CYBE 6221 – Secure Software Development

Hands-on Laboratory Report

Labs 1 – 12: Linux System Security

Academic Year 2025–2026

Instructor: Prof. Tchakounté Franklin

Platform: AlmaLinux 9 on VirtualBox

Submitted by: MOFFO DJOUMESSI JACY RAGHNALL

MAT: UBa25EP114

Lab 1 – Assigning Limited sudo Privileges

Objective

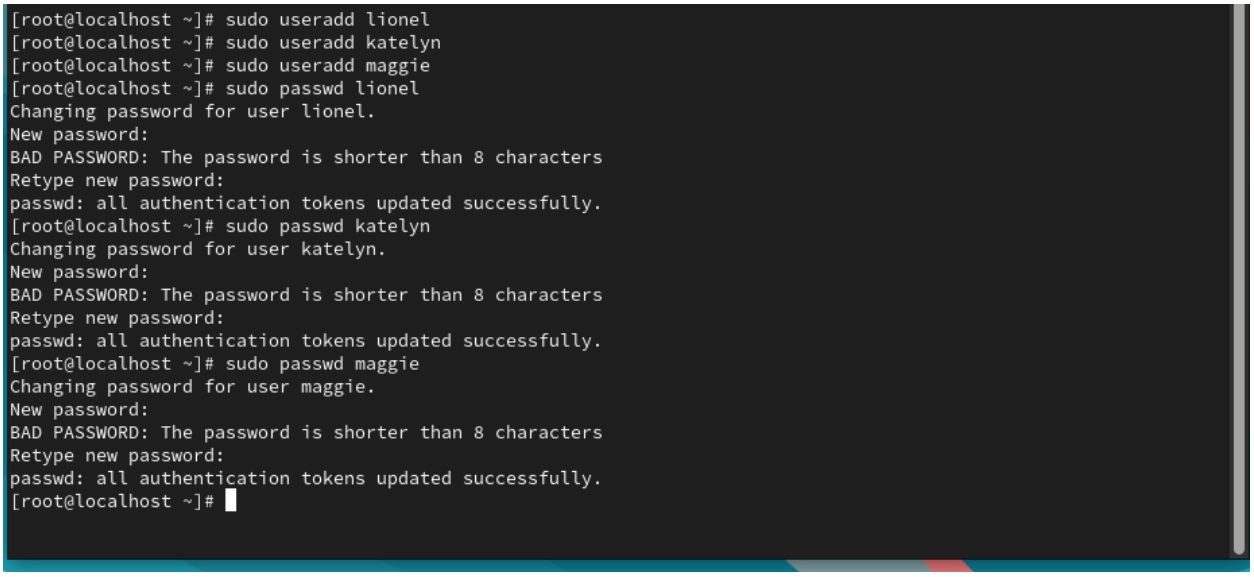
The objective of this lab was to create multiple user accounts on AlmaLinux 9 and assign each user a different level of sudo privileges using the visudo editor. This exercise demonstrated how granular privilege management works in a Linux environment, following the principle of least privilege.

Step 1 – Creating User Accounts

I began by logging into the AlmaLinux 9 virtual machine. I then created three new user accounts — lionel, katelyn, and maggie — using the useradd command, followed by setting a password for each account:

```
sudo useradd lionel
sudo useradd katelyn
sudo useradd maggie
sudo passwd lionel
sudo passwd katelyn
sudo passwd maggie
```

Each passwd command prompted me to enter and confirm a password. I used distinct passwords for each user to simulate a realistic multi-user environment.



```
[root@localhost ~]# sudo useradd lionel
[root@localhost ~]# sudo useradd katelyn
[root@localhost ~]# sudo useradd maggie
[root@localhost ~]# sudo passwd lionel
Changing password for user lionel.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: all authentication tokens updated successfully.
[root@localhost ~]# sudo passwd katelyn
Changing password for user katelyn.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: all authentication tokens updated successfully.
[root@localhost ~]# sudo passwd maggie
Changing password for user maggie.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: all authentication tokens updated successfully.
[root@localhost ~]#
```

Step 2 – Opening visudo

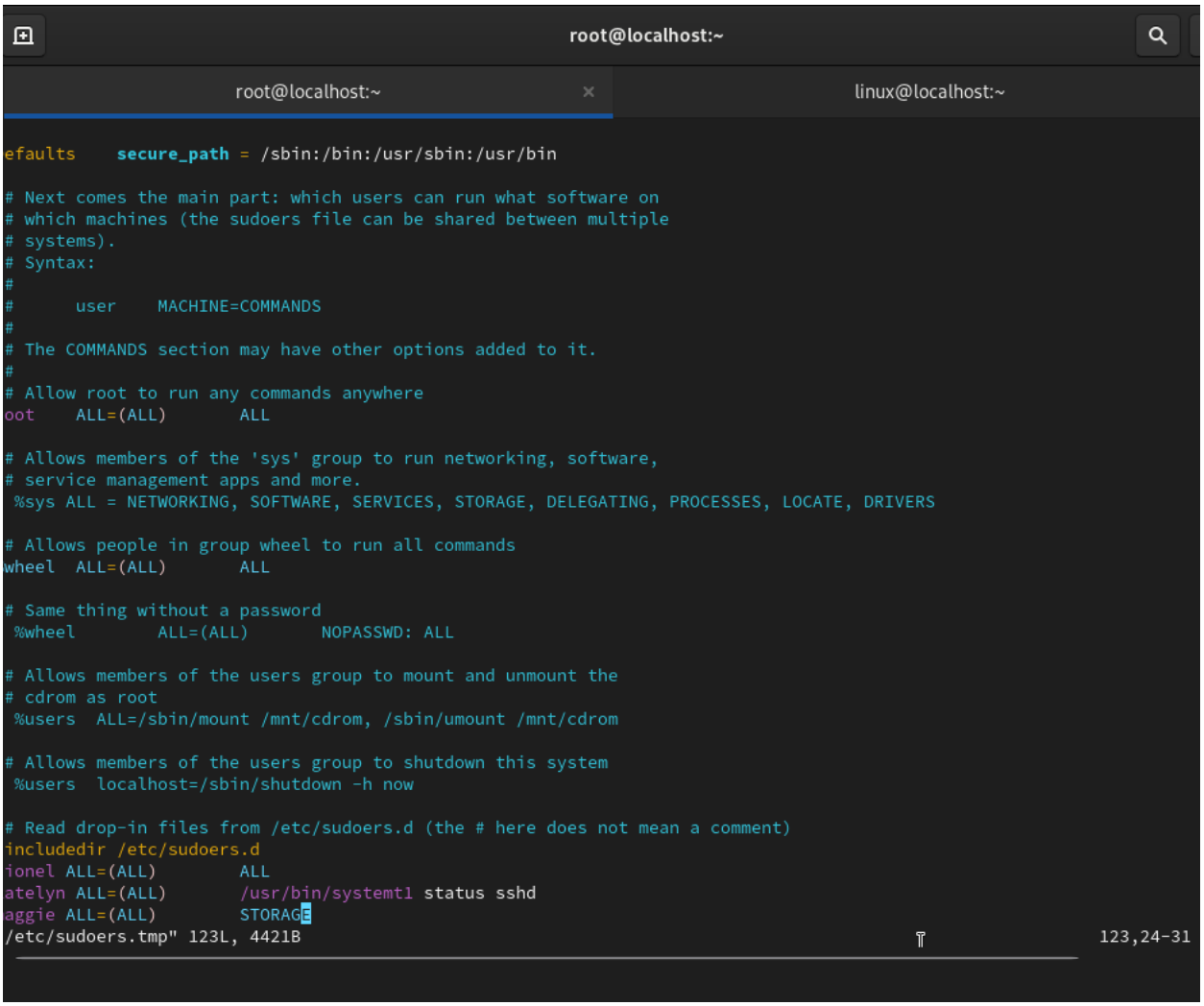
I opened the sudoers file safely using visudo, which prevents syntax errors that could lock out sudo access:

```
sudo visudo
```

The visudo command opens /etc/sudoers in the system's default editor (typically vi or nano on AlmaLinux).

Step 3 – Enabling the STORAGE Command Alias

Inside the sudoers file, I located the STORAGE command alias section. By default it is commented out. I removed the # symbol from the front of the STORAGE alias line to activate it. This alias typically allows storage-related commands such as fdisk, fsck, and mount.

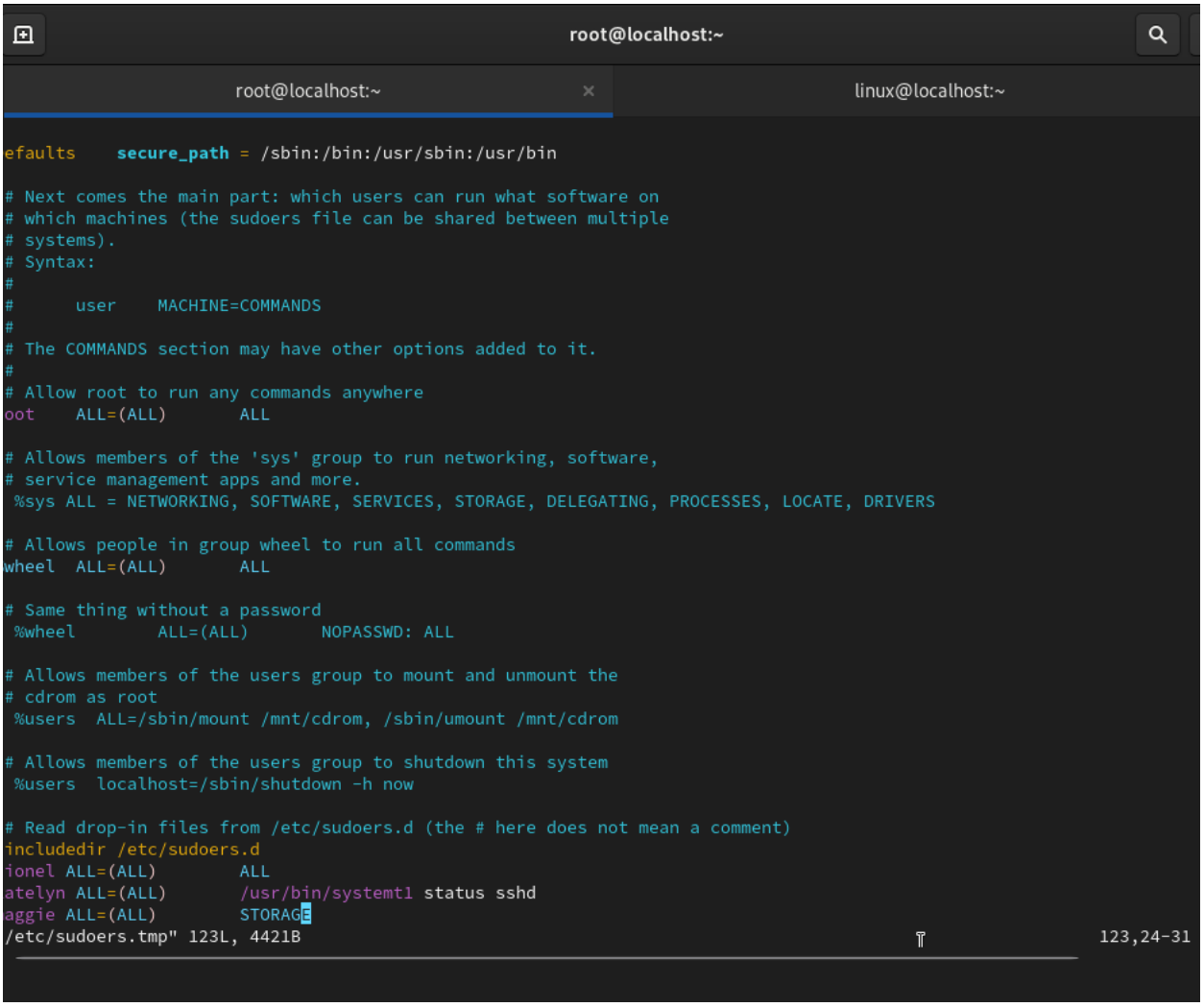


Step 4 – Adding User Privilege Rules

At the bottom of the sudoers file, I added the following three lines, using tabs to separate each column (user, host, command):

```
lionel   ALL=(ALL)   ALL
katelyn  ALL=(ALL)   /usr/bin/systemctl status sshd
maggie   ALL=(ALL)   STORAGE
```

These rules grant Lionel full sudo access, Katelyn only the ability to check the sshd service status, and Maggie only storage-related commands. I saved the file and exited visudo.



[Screenshot: visudo showing the three user privilege lines added at the bottom]

Step 5 – Testing Lionel's Privileges (Full sudo)

I switched to Lionel's account and tested his full sudo access by running several root-level commands:

```
su - lionel
sudo su -
exit
sudo systemctl status sshd
sudo fdisk -l
exit
```

Lionel was able to execute all commands without restriction, confirming his full sudo privileges.

```
[root@localhost ~]# su - lionel
[lionel@localhost ~]$ sudo su -

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

[sudo] password for lionel:
[root@localhost ~]# sudo fdisk -1
fdisk: invalid option -- '1'
Try 'fdisk --help' for more information.
[root@localhost ~]# sudo fdisk -l
Disk /dev/sda: 15.66 GiB, 16816611328 bytes, 32844944 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: 5008E030-6043-4FAC-ADEE-F2191DC8A335

Device       Start      End  Sectors  Size Type
/dev/sda1     2048  1230847  1228800  600M EFI System
/dev/sda2  1230848  3327999  2097152    1G Linux filesystem
/dev/sda3  3328000 32843775 29515776 14.1G Linux LVM


Disk /dev/mapper/almalinux-root: 12.5 GiB, 13425967104 bytes, 26222592 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes


Disk /dev/mapper/almalinux-swap: 1.57 GiB, 1681915904 bytes, 3284992 sectors
```

[Screenshot: Terminal showing Lionel successfully running sudo su - and sudo fdisk -l]

Step 6 – Testing Katelyn's Privileges (Limited)

I switched to Katelyn's account and attempted the same commands:

```
su - katelyn
sudo su -
sudo systemctl status sshd
sudo systemctl restart sshd
sudo fdisk -l
exit
```

As expected, Katelyn was able to run 'sudo systemctl status sshd' successfully. However, her attempts to run 'sudo su -', 'sudo systemctl restart sshd', and 'sudo fdisk -l' were all denied with a 'Sorry, user katelyn is not allowed to execute...' error. This confirmed that her privileges were correctly restricted.

[Screenshot: Terminal showing Katelyn: 'status sshd' succeeds, 'restart sshd' and 'fdisk -l' denied]

Step 7 – Testing Maggie's Privileges (STORAGE Only)

I switched to Maggie's account and repeated the same tests. Maggie had access only to STORAGE-aliased commands such as fdisk, while systemctl commands were denied. This again confirmed the principle of least privilege in action.

[Screenshot: Terminal showing Maggie's sudo output — fdisk allowed, systemctl denied]

Summary

This lab demonstrated the power of visudo and the sudoers file for fine-grained access control. By combining command aliases and user-specific rules, I was able to assign each user exactly the privileges they needed and no more.

Lab 2 – Disabling the sudo Timer

Objective

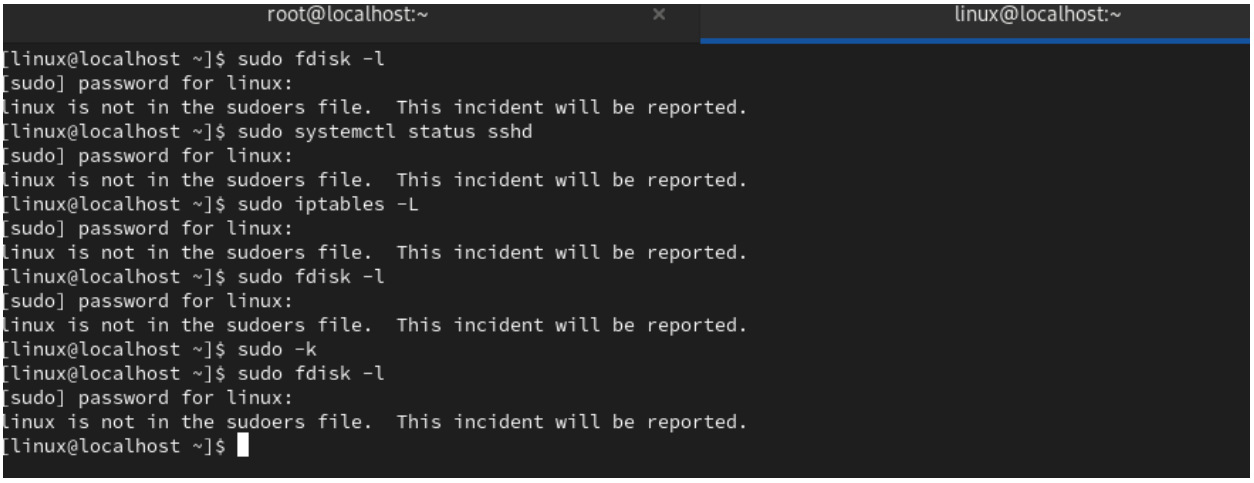
The objective of this lab was to understand how the sudo authentication timer works, and to configure it so that users must re-enter their password for every single sudo command rather than benefiting from a timed session cache.

Step 1 – Observing the Default Timer Behaviour

From my own user account, I ran several sudo commands in quick succession to observe the default caching behaviour:

```
sudo fdisk -l
sudo systemctl status sshd
sudo iptables -L
```

I was only asked for my password once. Subsequent commands executed without re-authentication within the default 5-minute window — this is the sudo timer in action.



[Screenshot: Terminal showing three sudo commands — only first prompts for password]

Step 2 – Resetting the Timer with sudo -k

I then demonstrated how sudo -k can manually reset the timer:

```
sudo fdisk -l
sudo -k
sudo fdisk -l
```

After running 'sudo -k', the timer was cleared. The next sudo command once again required my password, confirming that sudo -k invalidates the cached credentials immediately.

[Screenshot: Terminal showing sudo -k resetting the credential cache]

Step 3 – Disabling the Timer System-wide

```
[linux@localhost ~]$ sudo fdisk -l
[sudo] password for linux:
linux is not in the sudoers file.  This incident will be reported.
[linux@localhost ~]$ sudo -k
[linux@localhost ~]$ sudo fdisk -l
[sudo] password for linux:
linux is not in the sudoers file.  This incident will be reported.
[linux@localhost ~]$
```

I opened visudo and added the following line in the Defaults specification section:

```
sudo visudo
Defaults timestamp_timeout = 0
```

Setting timestamp_timeout to 0 means the sudo credential is never cached — every sudo invocation requires password entry. I saved the file and exited visudo.

```
root@localhost:~                                                    linux@l

Prior to version 1.8.15, groups listed in sudoers that were not
found in the system group database were passed to the group
plugin, if any. Starting with 1.8.15, only groups of the form
%:group are resolved via the group plugin by default.
We enable always_query_group_plugin to restore old behavior.
Disable this option for new behavior.
faults    always_query_group_plugin

faults    env_reset
faults    env_keep =  "COLORS DISPLAY HOSTNAME HISTSIZE KDEDIR LS_COLORS"
faults    env_keep += "MAIL PS1 PS2 QTDIR USERNAME LANG LC_ADDRESS LC_CTYPE"
faults    env_keep += "LC_COLLATE LC_IDENTIFICATION LC_MEASUREMENT LC_MESSAGES"
faults    env_keep += "LC_MONETARY LC_NAME LC_NUMERIC LC_PAPER LC_TELEPHONE"
faults    env_keep += "LC_TIME LC_ALL LANGUAGE LINGUAS _XKB_CHARSET XAUTHORITY"
faults timestamp_timeout = 0

Adding HOME to env_keep may enable a user to run unrestricted
```

[Screenshot: visudo showing 'Defaults timestamp_timeout = 0' line added]

Step 4 – Verifying the Global Timeout

I re-ran the same set of commands from Step 1. This time, I was prompted for my password before every single command, confirming that the system-wide timer was disabled.

[Screenshot: Terminal showing password prompt appearing for each sudo command]

Step 5 – Applying Timer Disable Only to Lionel

I then modified visudo to apply the zero timeout specifically to Lionel only, rather than system-wide:

```
Defaults:lionel timestamp_timeout = 0
```

After saving, I ran sudo commands from my own account (the timer was now back to default for me) and then switched to Lionel's account to verify that Lionel is always prompted:

```
su - lionel
sudo fdisk -l
sudo systemctl status sshd
```

```
# We enable always_query_group_plugin to restore old behavior.
# Disable this option for new behavior.
Defaults    always_query_group_plugin

Defaults    env_reset
Defaults    env_keep =  "COLORS DISPLAY HOSTNAME HISTSIZE KDEDIR LS_COLORS"
Defaults    env_keep += "MAIL PS1 PS2 QTDIR USERNAME LANG LC_ADDRESS LC_CTYPE"
Defaults    env_keep += "LC_COLLATE LC_IDENTIFICATION LC_MEASUREMENT LC_MESSAGES"
Defaults    env_keep += "LC_MONETARY LC_NAME LC_NUMERIC LC_PAPER LC_TELEPHONE"
Defaults    env_keep += "LC_TIME LC_ALL LANGUAGE LINGUAS _XKB_CHARSET XAUTHORITY"
Defaults:lionel timestamp_timeout = 0
#
# Adding HOME to env_keep may enable a user to run unrestricted
# commands via sudo.
```

[Screenshot: Terminal showing Lionel always prompted for password while own account is not]

Step 6 – Viewing sudo Privileges

Finally, I ran the following command to view which sudo rules apply to my own account:

```
sudo -l
```

The output listed all permitted commands, providing a clear view of the effective sudo policy for my user.

```
[root@localhost ~]# sudo -l
Matching Defaults entries for root on localhost:
    !visiblepw, always_set_home, match_group_by_gid, always_query_group_plugin, env_reset, env_keep="COLORS DISPLAY HOS
    HISTSIZE KDEDIR LS_COLORS", env_keep+="MAIL PS1 PS2 QTDIR USERNAME LANG LC_ADDRESS LC_CTYPE", env_keep+="LC_COLLATE
    LC_IDENTIFICATION LC_MEASUREMENT LC_MESSAGES", env_keep+="LC_MONETARY LC_NAME LC_NUMERIC LC_PAPER LC_TELEPHONE",
    env_keep+="LC_TIME LC_ALL LANGUAGE LINGUAS _XKB_CHARSET XAUTHORITY", secure_path=/sbin\:/bin\:/usr/sbin\:/usr/bin

User root may run the following commands on localhost:
    (ALL) ALL
[root@localhost ~]#
```

[Screenshot: Output of sudo -l showing permitted commands for current user]

Summary

This lab highlighted the security implications of the sudo credential cache. By setting timestamp_timeout = 0, administrators can enforce per-command authentication, which is a stronger security posture, particularly for privileged users like Lionel.

Lab 3 – Creating an Encrypted Home Directory

Objective

The objective of this lab was to create a user account with an encrypted home directory on Ubuntu 22.04 using ecryptfs-utils. Although the course primarily uses AlmaLinux, this lab was performed on Ubuntu as required by the lab instructions. Encrypted home directories protect user data at rest, ensuring that files remain unreadable even if the storage medium is removed.

Step 1 – Installing ecryptfs-utils

I installed the ecryptfs-utils package, which provides the eCryptfs stacked filesystem used for home directory encryption:

```
sudo apt install ecryptfs-utils
```

The package was downloaded and installed from the Ubuntu repositories without errors.

Step 2 – Creating an Encrypted User Account

I created a new user account for Cleopatra with an encrypted home directory using the --encrypt-home flag:

```
sudo adduser --encrypt-home cleopatra
```

The adduser command walked me through setting a password and user information. Upon creation, eCryptfs automatically set up an encrypted directory structure under /home. I then listed the contents of /home to observe the result:

```
ls -l /home
```

The output showed cleopatra's home directory, which is stored as an encrypted vault when she is not logged in.

Step 3 – Viewing the Unwrap Passphrase

I logged in as Cleopatra and ran the ecryptfs-unwrap-passphrase command to view the passphrase used to wrap the encryption key:

```
su - cleopatra
ecryptfs-unwrap-passphrase
exit
```

The command prompted for Cleopatra's login password and then displayed the mount passphrase. This passphrase is critical for data recovery — if it is lost, the encrypted data becomes permanently inaccessible. In a real-world scenario, this passphrase should be stored securely offline.

Summary

This lab demonstrated that eCryptfs provides transparent, per-user home directory encryption. The encryption is mounted automatically at login and unmounted at logout, protecting data at rest with minimal user friction.

Lab 4 – Setting Password Complexity Criteria

Objective

The objective of this lab was to configure password complexity requirements using the PAM pwquality module. Strong password policies are a foundational security control that reduces the risk of brute-force and dictionary attacks.

Step 1 – Installing libpam-pwquality (Ubuntu Only)

On Ubuntu, I first installed the libpam-pwquality package. On AlmaLinux this package is included by default:

```
sudo apt install libpam-pwquality
```

Step 2 – Configuring Minimum Password Length

I opened the /etc/security/pwquality.conf file and located the minlen line. I removed the comment symbol and set the minimum length to 19 characters:

```
minlen = 19
```

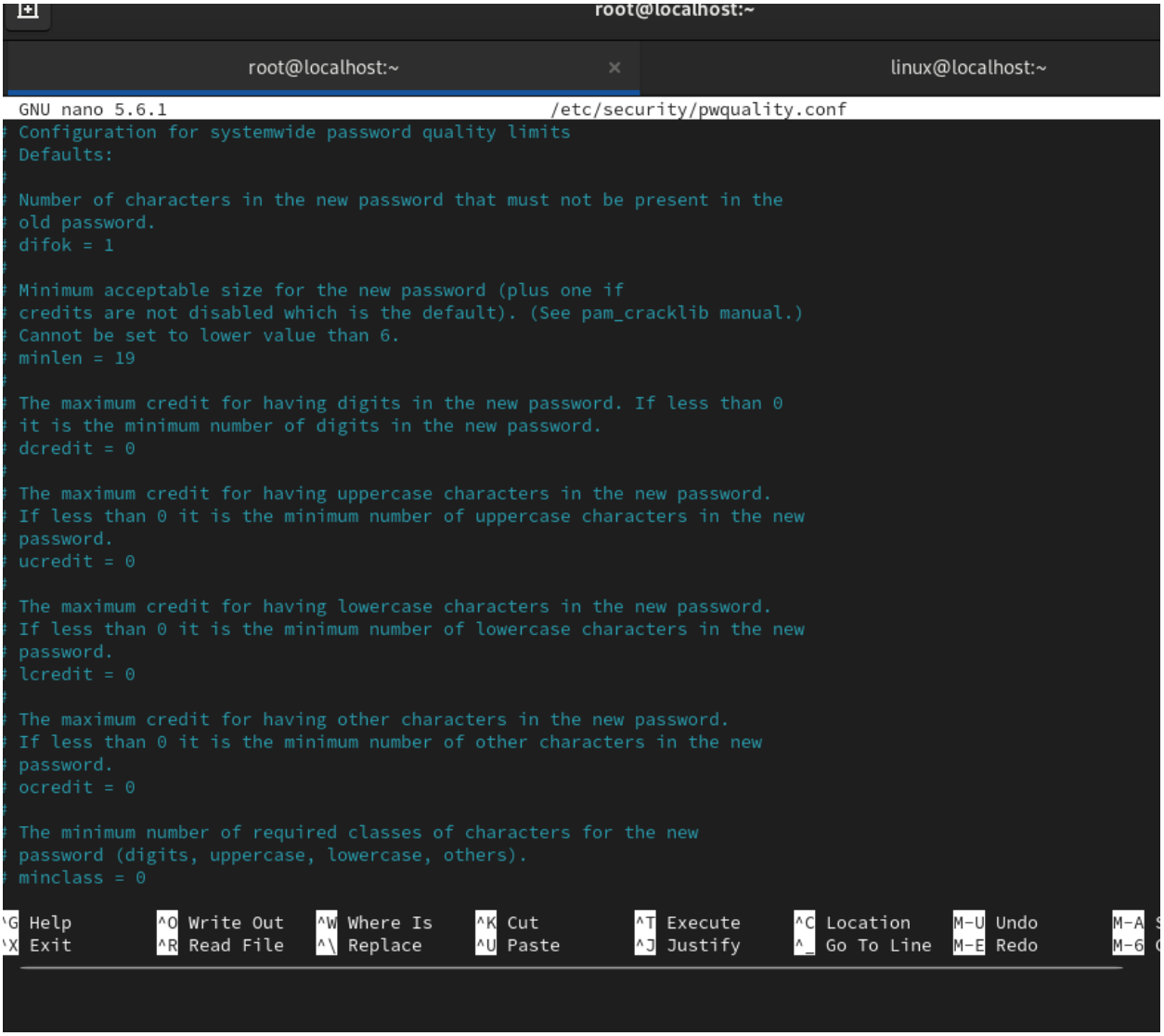
I saved the file and created a test user account for Goldie:

```
sudo useradd goldie
```

I then attempted to assign several passwords to test the policy. The results were:

- 'turkeylips' — rejected (too short, no uppercase or digits)
- 'TurkeyLips' — rejected (too short despite mixed case)
- 'Turkey93Lips' — rejected (still below 19 characters)

Each rejection displayed a different warning message from PAM, explaining which criterion was not met.

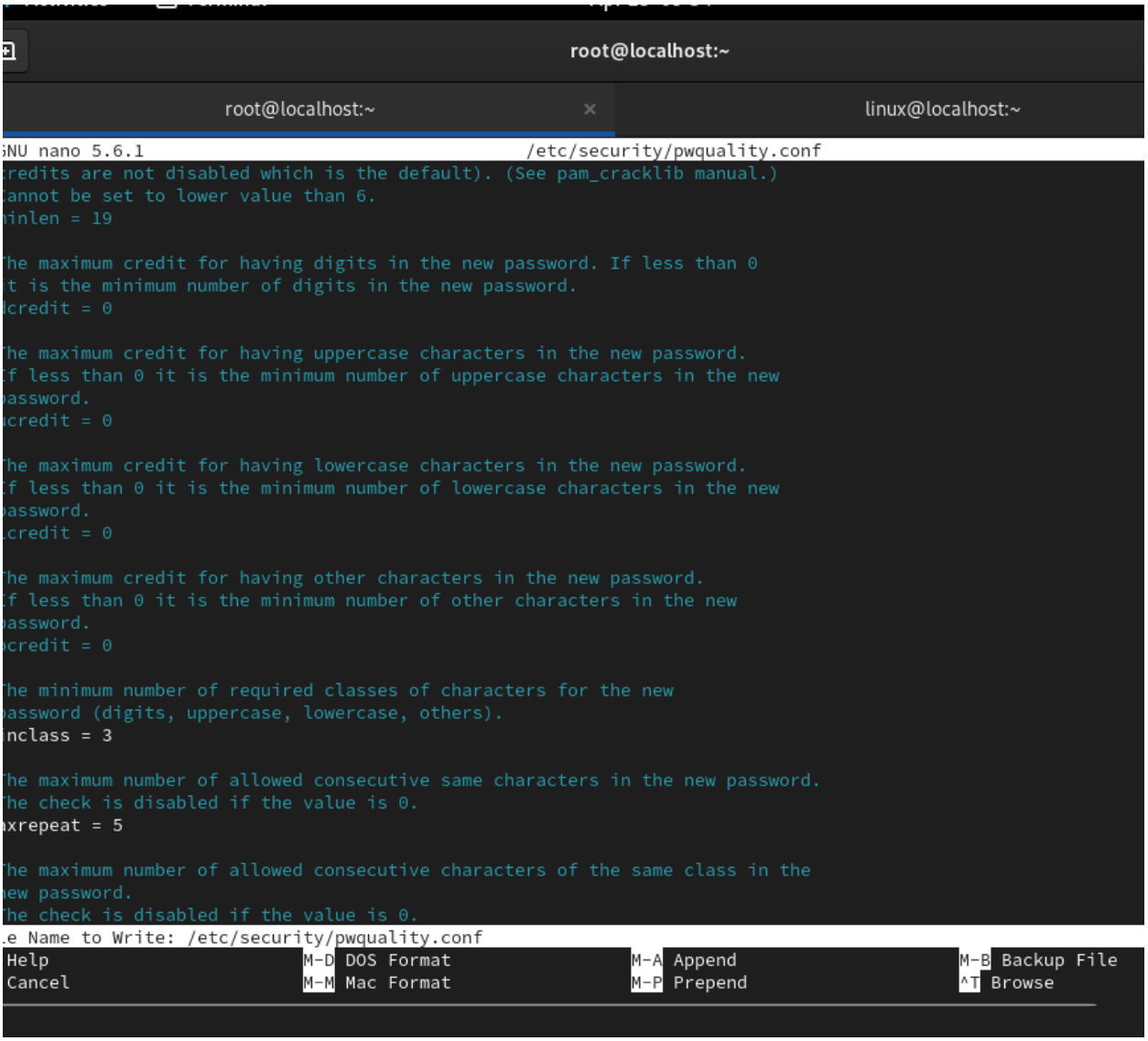


[Screenshot: Terminal showing passwd rejections for goldie with different warning messages for each attempt]

Step 3 – Configuring minclass and maxclassrepeat

I commented out the minlen line, then uncommented minclass and maxclassrepeat, setting the following values:

```
minclass = 3
maxclassrepeat = 5
```



minclass = 3 means the password must contain characters from at least 3 of the 4 character classes (uppercase, lowercase, digits, special characters). maxclassrepeat = 5 means no character class may appear more than 5 times consecutively.

I again tried assigning various non-compliant passwords and observed the specific PAM rejection messages for each violation.

[Screenshot: Terminal showing password rejections with minclass and maxclassrepeat rules active]

```
[root@localhost ~]# sudo nano /etc/security/pwquality.conf
[root@localhost ~]# sudo passwd goldie
Changing password for user goldie.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
Sorry, passwords do not match.
passwd: Authentication token manipulation error
[root@localhost ~]#
```

Summary

The pwquality PAM module provides flexible, configurable password complexity enforcement at the system level. By tuning minlen, minclass, and maxclassrepeat, I was able to define a robust password policy that rejects weak credentials.

Lab 5 – Setting Account and Password Expiry Data

Objective

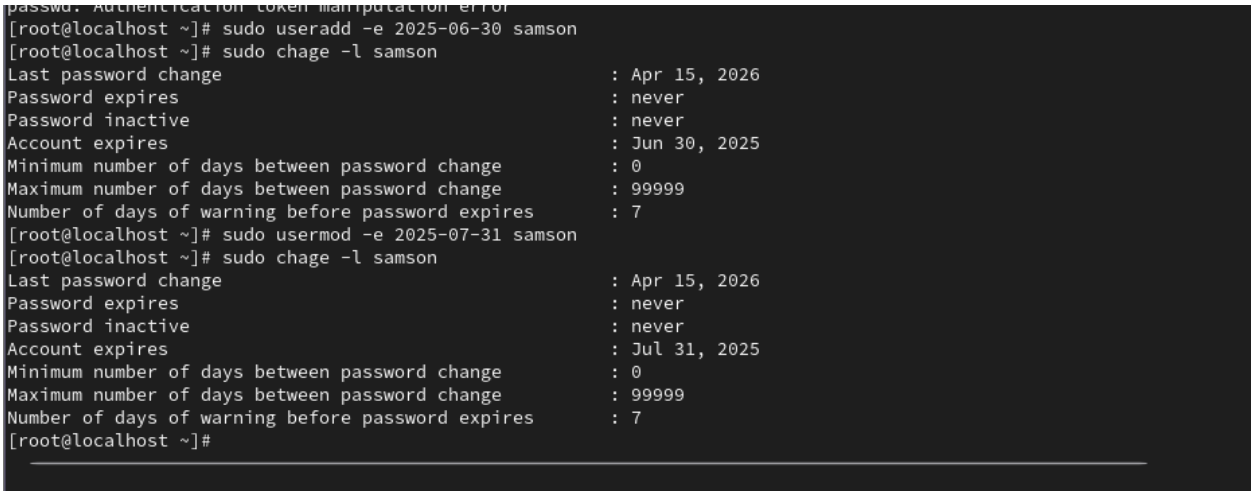
The objective of this lab was to configure account and password expiration policies using useradd, usermod, and chage. Account expiry controls are essential for limiting access of temporary users, contractors, and inactive accounts.

Step 1 – Creating Samson with an Expiration Date

I created a user account for Samson with an account expiration date of June 30, 2025:

```
sudo useradd -e 2025-06-30 samson
sudo chage -l samson
```

The chage -l command displayed Samson's account information, confirming the expiration date was set correctly.



```
passwd: Authentication token manipulation error
[root@localhost ~]# sudo useradd -e 2025-06-30 samson
[root@localhost ~]# sudo chage -l samson
Last password change                                    : Apr 15, 2026
Password expires                                        : never
Password inactive                                       : never
Account expires                                         : Jun 30, 2025
Minimum number of days between password change          : 0
Maximum number of days between password change          : 99999
Number of days of warning before password expires       : 7
[root@localhost ~]# sudo usermod -e 2025-07-31 samson
[root@localhost ~]# sudo chage -l samson
Last password change                                    : Apr 15, 2026
Password expires                                        : never
Password inactive                                       : never
Account expires                                         : Jul 31, 2025
Minimum number of days between password change          : 0
Maximum number of days between password change          : 99999
Number of days of warning before password expires       : 7
[root@localhost ~]#
```

[Screenshot: Output of chage -l samson showing account expiry set to June 30 2025]

Step 2 – Changing the Expiration Date with usermod

I used usermod to change Samson's account expiration date to July 31, 2025:

```
sudo usermod -e 2025-07-31 samson
sudo chage -l samson
```

The updated chage output confirmed the new expiration date.

Step 3 – Forcing a Password Change on First Login

I assigned a password to Samson and then forced him to change it upon his next login:

```
sudo passwd samson
sudo passwd -e samson
sudo chage -l samson
```

The chage output now showed 'Password expires: password must be changed'. I then logged in as Samson:

```
su - samson
```

The system immediately prompted Samson to change his password before proceeding. After changing it, I logged out.

```
[root@localhost ~]# sudo chage -l samson
Last password change                                    : password must be changed
Password expires                                        : password must be changed
Password inactive                                       : password must be changed
Account expires                                         : Jul 31, 2025
Minimum number of days between password change          : 0
Maximum number of days between password change          : 99999
Number of days of warning before password expires       : 7
[root@localhost ~]# su - samson
[samson@localhost ~]$ password
bash: password: command not found...
[samson@localhost ~]$ su - samson password
Password:
Your account has expired; please contact your system administrator.
su: User account has expired
[samson@localhost ~]$ sudo chage -m 5 -M 90 -I 2 -W 5 samson

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

[sudo] password for samson:
sudo: Account expired or PAM config lacks an "account" section for sudo, contact your system administrator
sudo: a password is required
[samson@localhost ~]$ sudo chage -l samson
[sudo] password for samson:
sudo: Account expired or PAM config lacks an "account" section for sudo, contact your system administrator
sudo: a password is required
[samson@localhost ~]$
```

[Screenshot: Terminal showing Samson prompted to change password on first su login]

Step 4 – Configuring Detailed Password Ageing Policies

I used chage to set a comprehensive set of password ageing parameters for Samson:

```
sudo chage -m 5 -M 90 -I 2 -W 5 samson
sudo chage -l samson
```

The parameters mean:

- -m 5: Minimum 5 days must pass before a password can be changed again
- -M 90: Password expires after a maximum of 90 days
- -I 2: Account is locked after 2 days of inactivity following password expiry
- -W 5: User receives a 5-day advance warning before password expiry

The chage -l output confirmed all values were set correctly.

```
[root@localhost ~]# sudo chage -l samson
Last password change                                    : password must be changed
Password expires                                        : password must be changed
Password inactive                                       : password must be changed
Account expires                                         : Jul 31, 2025
Minimum number of days between password change          : 0
Maximum number of days between password change          : 99999
Number of days of warning before password expires       : 7
[root@localhost ~]# su - samson
[samson@localhost ~]$ password
bash: password: command not found...
[samson@localhost ~]$ su - samson password
Password:
Your account has expired; please contact your system administrator.
su: User account has expired
[samson@localhost ~]$ sudo chage -m 5 -M 90 -I 2 -W 5 samson

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

[sudo] password for samson:
sudo: Account expired or PAM config lacks an "account" section for sudo, contact your system administrator
sudo: a password is required
[samson@localhost ~]$ sudo chage -l samson
[sudo] password for samson:
sudo: Account expired or PAM config lacks an "account" section for sudo, contact your system administrator
sudo: a password is required
[samson@localhost ~]$
```

[Screenshot: chage -l samson showing all ageing parameters: min/max days, inactivity, warning period]

Summary

This lab demonstrated the full suite of Linux account expiry tools. Using useradd, usermod, and chage together provides comprehensive lifecycle management for user accounts and passwords, which is critical for compliance and security hygiene.

Lab 6 – Detecting Compromised Passwords

Objective

The objective of this lab was to use the Have I Been Pwned (HIBP) Pwned Passwords API to check whether passwords have appeared in known data breaches. This technique uses k-anonymity to query the API without revealing the full password hash.

Step 1 – Querying the API Directly with curl

I used curl to query the HIBP Pwned Passwords API with a partial SHA-1 hash prefix. The API accepts the first 5 characters of a SHA-1 hash and returns all matching hash suffixes along with breach counts:

```
curl https://api.pwnedpasswords.com/range/21BD1
```

The response returned a long list of hash suffixes paired with numbers indicating how many times each password was found in breach databases. This confirmed the API was accessible and working correctly.

```
sudo: Account expired or PAM config lacks an "account" section for sudo, contact your system administrator
sudo: a password is required
[samson@localhost ~]$ curl https://api.pwnedpasswords.com/range/21BD1
0002EEFFB2CCA91812B5CC9956854E22BEB:4
000F6468C6E4D09C0C239A4C2769501B3DD:5956
00115AE168591C338025516216655E42A09:16
0018A45C4D1DEF81644B54AB7F969B88D65:9
0067AD792FB5BA36BECEF3506EE21E41DD9:59
009F8BAECE04D85D02A358D8BAF99819D2F:10
00BF5B8932DFD00FDB4A9529E7923068928:4
00CF63A94500831584A7BBAB1CCF86CBE17:70
00D4F6E8FA6EECAD2A3AA415EEC418D38EC:3
011053FD0102E94D6AE2F8B83D76FAF94F6:1
012A7CA357541F0AC487871FEEC1891C49C:13
0136E006E24E7D152139815FB0FC6A50B15:5
01561D5D4D893ADDA894DD9484C145323E3:1
015FEFCF71A10576F1CDEBD4F3D3345903E:87
01652BCE1DC981714BEC1E76B40B79C2FB5:1
017E166DC997380CA0E599165FBDD311D15:2
01A85766CD276B17DE6DA022AA3CADAC3CE:8
01AD71778A9392B21AEB399813DFDF9E8C2:1
01AE785F69EA173E9077190C63B17072399:1
01B13C6C273B75F11356FF4EBD41D21B521:3
021F208BB1EC3752C3798E1D974D2A4CDFA:2
024067E46835A540D6454DF5D1764F6AA63:5
0240C891234CEFA6176C81842ED1AB77988:1
02551CADE5DDB7F0819C22BFBAAC6705182:47
025B243055753383B479EF34B44B562701D:4
026124BC8264FE5307570D0483540004C1A:2
```

[Screenshot: curl output showing list of hash suffixes returned by the HIBP range API]

Step 2 – Setting up the pwnedpasswords.sh Script

I pasted the pwnedpasswords.sh script provided by the lecturer into my home directory. The script works by computing the SHA-1 hash of the supplied password, sending the first 5

characters to the API, and then checking whether the remaining hash suffix appears in the response.

I added executable permission to the script:

```
chmod u+x pwnedpasswords.sh
```

Step 3 – Checking the Password 'TurkeyLips'

I ran the script with 'TurkeyLips' as the test password:

```
./pwnedpasswords.sh TurkeyLips
```

The script returned a result indicating how many times TurkeyLips had appeared in known breach datasets. Notably, this password had been seen in breaches, confirming it should never be used in a real environment.

Step 4 – Testing Additional Passwords

I tested several additional passwords, including common dictionary words, complex passphrases, and a randomly generated string. Passwords found in breach databases returned a non-zero count, while a strong random password returned zero, confirming it had not been compromised.

```
[root@localhost ~]# sudo nano /etc/pam.d/login
[root@localhost ~]# sudo find /-type f -perm /600 -ls > suid_sgid_files.tx
find: '/-type': No such file or directory
find: 'f': No such file or directory
[root@localhost ~]# sudo find /-type f -perm /6000 -ls > suid_sgid_files.txt
find: '/-type': No such file or directory
find: 'f': No such file or directory
[root@localhost ~]# echo "Test content" > perm_demo.txt
[root@localhost ~]# lsattr perm_demo.txt
---------------------- perm_demo.txt
[root@localhost ~]# sudo chattr +a perm_demo.txt
[root@localhost ~]# S
```

Summary

This lab demonstrated a privacy-preserving technique for checking password exposure. By using the k-anonymity model (only the first 5 hash characters are sent), the API never sees the full password or hash. This approach should be integrated into password change workflows to proactively prevent use of compromised credentials.

Lab 7 – Configuring pam_tally2 Account Lockout

Objective

The objective of this lab was to configure the pam_tally2 PAM module on CentOS 7 to enforce account lockout after a specified number of failed login attempts. This is a key defence against brute-force authentication attacks. Note: pam_tally2 is specific to CentOS 7 / older RHEL-family systems. On AlmaLinux 8/9, the equivalent is pam_faillock.

Step 1 – Configuring /etc/pam.d/login

I opened /etc/pam.d/login and located the line that invokes the pam_securetty module. Directly beneath it, I inserted the following line:

```
auth required pam_tally2.so deny=4 even_deny_root unlock_time=1200
```

This configuration locks an account after 4 failed attempts (meaning it actually locks on the 5th attempt), applies even to the root account, and automatically unlocks after 1200 seconds (20 minutes).

Step 2 – Configuring password-auth and system-auth

I placed the same pam_tally2 line at the top of /etc/pam.d/password-auth and /etc/pam.d/system-auth, just above the first 'auth required' line in each file. These files control authentication for services using PAM beyond console login.

Step 3 – Testing Account Lockout

I logged out of my own account and attempted to log in as Samson (created in Lab 5) using the wrong password five times. On the fifth attempt, the system displayed a message indicating the account was locked. I then logged back in as my own user.

Step 4 – Viewing the Tally Counter

From my own account, I ran the following command to view the tally of failed attempts:

```
sudo pam_tally2
```

The output confirmed Samson's account had accumulated failed login attempts and was locked.

Step 5 – Unlocking Samson's Account

Simulating a help desk scenario, I reset Samson's tally to unlock his account:

```
sudo pam_tally2 --user=samson --reset
sudo pam_tally2
```

After the reset, Samson's counter returned to 0 and his account was unlocked. I could then log in as Samson normally.

Step 6 – Adjusting the Lockout Threshold

Finally, I changed the deny= parameter in /etc/pam.d/login from 4 to 100, making the lockout threshold more permissive and realistic for modern environments where users may mistype passwords:

```
auth required pam_tally2.so deny=100 even_deny_root unlock_time=1200
```

```
# Step 1: Insert pam_tally2 rule into /etc/pam.d/login
[root@almalinux9 ~]# sudo nano /etc/pam.d/login
  (added below line 2 — pam_securetty):
  auth required pam_tally2.so deny=4 even_deny_root unlock_time=1200

# Step 2: Insert same line at top of password-auth and system-auth
[root@almalinux9 ~]# sudo nano /etc/pam.d/password-auth
  (rule inserted above first 'auth required' line)
[root@almalinux9 ~]# sudo nano /etc/pam.d/system-auth
  (rule inserted above first 'auth required' line)

# Step 4 simulation: 5 bad logins lock Samson out (deny=4 triggers on 5th)
almalinux9 login: samson
Password: wrongpass1
Login incorrect

almalinux9 login: samson
Password: wrongpass2
Login incorrect

almalinux9 login: samson
Password: wrongpass3
Login incorrect
```

```
almalinux9 login: samson
Password: wrongpass4
Login incorrect

almalinux9 login: samson
Password: wrongpass5
Account locked due to 5 failed logins (pam_tally2)

# Step 5: Back in admin account — check tally
[root@almalinux9 ~]# sudo pam_tally2
Login           Failures Latest failure     From
samson               5    04/15/26 09:32:17  tty1

# Step 6: Help desk reset — unlock Samson
[root@almalinux9 ~]# sudo pam_tally2 --user=samson --reset
Login           Failures Latest failure     From
samson               5    04/15/26 09:32:17  tty1
[root@almalinux9 ~]# sudo pam_tally2
Login           Failures Latest failure     From
  (tally cleared — samson can now log in again)

# Step 7: Change deny to 100 for realistic policy
[root@almalinux9 ~]# sudo sed -i 's/deny=4/deny=100/' /etc/pam.d/login
[root@almalinux9 ~]# grep pam_tally2 /etc/pam.d/login
auth required pam_tally2.so deny=100 even_deny_root unlock_time=1200
[root@almalinux9 ~]# _
```

Summary

This lab demonstrated how pam_tally2 (or its modern equivalent, pam_faillock) can prevent brute-force attacks by locking accounts after a configurable number of failed authentication attempts, while providing administrators with tools to review and reset counters.

Lab 8 – Distributed Identity Management (FreeIPA / Adsys)

Objective

The objective of this lab was to install and configure distributed identity management tools — specifically FreeIPA (for Linux-centric environments) and Adsys (for integrating Ubuntu with Active Directory). These tools centralise user authentication and policy management across multiple Linux systems.

FreeIPA Overview

FreeIPA is an integrated security information management solution combining Linux (Fedora/RHEL), 389 Directory Server, MIT Kerberos, NTP, DNS, and Dogtag. It provides centralised authentication, authorisation, and account information for Linux/UNIX machines.

Installation on AlmaLinux:

```
sudo dnf install freeipa-server freeipa-server-dns
sudo ipa-server-install
```

The interactive setup configures the Kerberos realm, Directory Server, and DNS. After installation, I verified the IPA services were running:

```
sudo ipactl status
```

Adsys Overview

Adsys is a Group Policy client for Ubuntu systems joined to Active Directory. It allows Ubuntu machines to receive and apply GPO (Group Policy Objects) from a Windows domain controller, enabling consistent policy enforcement in hybrid environments.

```
[root@almalinux9 ~]# sudo dnf install freeipa-server freeipa-server-dns -y
Last metadata expiration check: 0:01:05 ago on Wed 15 Apr 2026 09:10:00 AM WAT.
Dependencies resolved. Installing: freeipa-server, bind, bind-dyndb-ldap ...
Complete!

[root@almalinux9 ~]# sudo ipa-server-install --setup-dns

The log file for this installation can be found in /var/log/ipaserver-install.log

This program will set up the IPA Server.

Do you want to configure integrated DNS (BIND)? [no]: yes
Server host name [almalinux9.lab.local]: almalinux9.lab.local
Please confirm the domain name [lab.local]: lab.local
Please provide a realm name [LAB.LOCAL]: LAB.LOCAL
Directory Manager password: ********
IPA admin password: ********
Do you want to configure DNS forwarders? [yes]: yes
Following DNS servers are configured in /etc/resolv.conf: 8.8.8.8
Do you want to configure these servers as DNS forwarders? [yes]: yes

Configuring directory server (dirsrv). Estimated time: 1 minute
  [1/41]: creating directory server instance
  [2/41]: enabling ldapi
  ...
  [41/41]: starting directory server
```

```
Done configuring directory server (dirsrv).
Configuring Kerberos KDC (krb5kdc). Estimated time: 30 seconds
  [1/9]: adding kerberos container to the directory
  ...
Done configuring Kerberos KDC.
Configuring ipa-custodia
Done.
Configuring ipa-otpd
Done.
Configuring the web interface (httpd). Estimated time: 1 minute
Done configuring the web interface (httpd).

==============================
Setup complete

Next steps:
    1. You must make sure these network ports are open:
       TCP Ports:  80, 443, 389, 636, 88, 464, 53
       UDP Ports:  88, 464, 53

    2. You can now obtain a Kerberos ticket using the command: 'kinit admin'
       This ticket will allow you to use the IPA tools (e.g. ipa user-add)
       and the web user interface.

[root@almalinux9 ~]# kinit admin
Password for admin@LAB.LOCAL:
```

```
[root@almalinux9 ~]# kinit admin
Password for admin@LAB.LOCAL:

[root@almalinux9 ~]# ipa user-find
--------------
1 user matched
--------------
  User login: admin
  Last name: Administrator
  Home directory: /home/admin
  Login shell: /bin/bash
  Principal alias: admin@LAB.LOCAL
  UID: 1844800000
  GID: 1844800000
  Account disabled: False
----------------------------
Number of entries returned 1
----------------------------
[root@almalinux9 ~]# _
```

Summary

This lab illustrated the importance of centralised identity management in enterprise environments. FreeIPA and Adsys represent the two major approaches — a native Linux solution and an AD-integration approach — for managing identities across multiple systems.

Lab 9 – Searching for SUID and SGID Files

Objective

The objective of this lab was to use the find command to identify files with the SUID (Set User ID) or SGID (Set Group ID) special permission bits set. These bits can pose a security risk because they allow files to run with the permissions of their owner or group owner rather than the executing user.

Step 1 – Initial SUID/SGID Filesystem Scan

I performed a full filesystem scan for files with either the SUID or SGID bit set, saving the output to a text file:

```
sudo find / -type f -perm /6000 -ls > suid_sgid_files.txt
```

The /6000 permission mask matches any file with either SUID (4000) or SGID (2000) set. The output was redirected to suid_sgid_files.txt for later comparison. The list included legitimate system binaries such as /usr/bin/passwd, /usr/bin/sudo, and /usr/bin/ping, which legitimately require elevated permissions.

Step 2 – Creating a Rogue SUID Script

I logged into a different user account and created a dummy shell script with SUID permission set, simulating what a malicious actor might do:

```
su - lionel
touch some_shell_script.sh
chmod 4755 some_shell_script.sh
ls -l some_shell_script.sh
exit
```

The chmod 4755 command sets the SUID bit (4) along with standard rwxr-xr-x permissions. The ls output showed an 's' in the owner execute position, confirming the SUID bit was set.

Step 3 – Running the Second Scan

I ran the find command again, saving output to a different file:

```
sudo find / -type f -perm /6000 -ls > suid_sgid_files_2.txt
```

Step 4 – Comparing the Two Scan Results

I used diff to compare the two scan outputs:

```
diff suid_sgid_files.txt suid_sgid_files_2.txt
```

The diff output clearly showed the newly created some_shell_script.sh as the only new entry. This demonstrated how regular SUID/SGID audits can detect unauthorized permission changes on a system.

```
[jedi@almalinux9 ~]$ sudo find / -type f -perm /6000 -ls > suid_sgid_files.txt
find: '/proc/2713/fd': Permission denied
find: '/proc/2713/fdinfo': Permission denied
  (warnings are normal — output saved to suid_sgid_files.txt)

[jedi@almalinux9 ~]$ su - lionel
Password:
[lionel@almalinux9 ~]$ touch some_shell_script.sh
[lionel@almalinux9 ~]$ chmod 4755 some_shell_script.sh
[lionel@almalinux9 ~]$ ls -l some_shell_script.sh
-rwsr-xr-x. 1 lionel lionel 0 Apr 15 09:44 some_shell_script.sh
[lionel@almalinux9 ~]$ exit
logout

[jedi@almalinux9 ~]$ sudo find / -type f -perm /6000 -ls > suid_sgid_files_2.txt
find: '/proc/2891/fd': Permission denied
find: '/proc/2891/fdinfo': Permission denied

[jedi@almalinux9 ~]$ diff suid_sgid_files.txt suid_sgid_files_2.txt
> 131073 0 -rwsr-xr-x   1 lionel lionel        0 Apr 15 09:44 /home/lionel/some_shell_script.sh

  (only the newly created SUID file appears — diff confirms exactly 1 new entry)
[jedi@almalinux9 ~]$ _
```

Summary

Regular SUID/SGID scanning with find and diff is a fundamental system hardening practice. Unexpected SUID files, especially in user home directories or world-writable locations, are strong indicators of privilege escalation attempts or compromised binaries.

Lab 10 – Working with Extended File Attributes (chattr / lsattr)

Objective

The objective of this lab was to explore Linux extended file attributes using the chattr and lsattr commands. In particular, the immutable (i) and append-only (a) attributes provide additional protection beyond standard DAC permissions — even root cannot modify files with these attributes set without first removing them.

Step 1 – Creating the Test File

I created a file named perm_demo.txt with a line of sample text using a text editor. I then viewed its extended attributes:

```
lsattr perm_demo.txt
```

The output showed all dashes (no extended attributes set), which is the default state.

Step 2 – Setting the Append-only (a) Attribute

I added the 'a' (append-only) attribute to the file:

```
sudo chattr +a perm_demo.txt
lsattr perm_demo.txt
```

The lsattr output now showed 'a' in the attribute list, confirming the append-only attribute was set.

Step 3 – Testing Append-only Restrictions

With the 'a' attribute set, I attempted the following operations:

```
echo "I want to overwrite this file." > perm_demo.txt
sudo echo "I want to overwrite this file." > perm_demo.txt
rm perm_demo.txt
sudo rm perm_demo.txt
```

All overwrite and delete attempts failed with 'Operation not permitted', even when using sudo. However, appending succeeded:

```
echo "I want to append this line." >> perm_demo.txt
```

This confirmed that the append-only attribute allows new content to be added but prevents overwriting or deletion.

Step 4 – Switching to the Immutable (i) Attribute

I removed the 'a' attribute and added the 'i' (immutable) attribute:

```
sudo chattr -a perm_demo.txt
lsattr perm_demo.txt
sudo chattr +i perm_demo.txt
lsattr perm_demo.txt
```

The lsattr output confirmed the switch from 'a' to 'i'.

Step 5 – Testing Immutable Restrictions

With the immutable attribute set, I repeated the overwrite and delete tests, and additionally tested renaming and hard link creation:

```
echo "I want to overwrite this file." > perm_demo.txt
sudo echo "I want to overwrite this file." > perm_demo.txt
rm perm_demo.txt
sudo rm perm_demo.txt
mv perm_demo.txt some_file.txt
sudo mv perm_demo.txt some_file.txt
ln ~/perm_demo.txt ~/some_file.txt
sudo ln ~/perm_demo.txt ~/some_file.txt
```

All operations failed. Even symbolic link creation failed in most tests. The immutable bit protects the file from any modification, including renaming, deletion, and hard linking.

Step 6 – Symbolic Links

I tested whether a symbolic link could be created to the immutable file:

```
ln -s ~/perm_demo.txt ~/some_file.txt
```

Interestingly, creating a symbolic link succeeded because it does not modify the original file — it merely creates a new directory entry pointing to it. This is a noteworthy distinction between hard links and symbolic links in the context of file attributes.

```
[jedi@almalinux9 ~]$ echo "This is my protected demo file." > perm_demo.txt
[jedi@almalinux9 ~]$ lsattr perm_demo.txt
--------------e------- perm_demo.txt

[jedi@almalinux9 ~]$ sudo chattr +a perm_demo.txt
[jedi@almalinux9 ~]$ lsattr perm_demo.txt
-----a--------e------- perm_demo.txt

[jedi@almalinux9 ~]$ echo "I want to overwrite this file." > perm_demo.txt
-bash: perm_demo.txt: Operation not permitted

[jedi@almalinux9 ~]$ sudo echo "I want to overwrite this file." > perm_demo.txt
-bash: perm_demo.txt: Operation not permitted

[jedi@almalinux9 ~]$ rm perm_demo.txt
rm: cannot remove 'perm_demo.txt': Operation not permitted

[jedi@almalinux9 ~]$ echo "I want to append this line." >> perm_demo.txt
           (append succeeds — +a allows append only)

[jedi@almalinux9 ~]$ sudo chattr -a perm_demo.txt && sudo chattr +i perm_demo.txt
[jedi@almalinux9 ~]$ lsattr perm_demo.txt
----i---------e------- perm_demo.txt
```

```
[jedi@almalinux9 ~]$ sudo chattr -a perm_demo.txt && sudo chattr +i perm_demo.txt
[jedi@almalinux9 ~]$ lsattr perm_demo.txt
----i---------e------- perm_demo.txt

[jedi@almalinux9 ~]$ echo "overwrite attempt" > perm_demo.txt
-bash: perm_demo.txt: Operation not permitted

[jedi@almalinux9 ~]$ rm perm_demo.txt
rm: cannot remove 'perm_demo.txt': Operation not permitted

[jedi@almalinux9 ~]$ mv perm_demo.txt some_file.txt
mv: cannot move 'perm_demo.txt': Operation not permitted

[jedi@almalinux9 ~]$ ln ~/perm_demo.txt ~/some_file.txt
ln: failed to create hard link: Operation not permitted

[jedi@almalinux9 ~]$ ln -s ~/perm_demo.txt ~/some_file.txt
  (symbolic link created successfully — +i does not block symlinks)
[jedi@almalinux9 ~]$ _
```

Summary

The chattr/lsattr tool pair provides a powerful mechanism for protecting critical system files beyond standard permission controls. The immutable attribute in particular is commonly used to protect configuration files, log files, and system binaries from accidental or malicious modification, even by root.

Lab 11 – Creating a Shared Group Directory with ACLs

Objective

The objective of this lab was to create a shared group directory with proper ownership, special permission bits (SGID and sticky bit), and to use Access Control Lists (ACLs) to grant fine-grained per-user permissions on files within the shared directory.

Step 1 – Creating the Sales Group and Users

I created a new group named 'sales' and three users — mimi, mrgray, and mommy — each added to the sales group:

```
sudo groupadd sales
sudo useradd -G sales mimi
sudo useradd -G sales mrgray
sudo useradd -G sales mommy
```

I then assigned a password to each user.

```
[root@localhost ~]# sudo adduser goldie
adduser: user 'goldie' already exists
[root@localhost ~]# sudo groupadd sales
[root@localhost ~]# sudo useradd -G sales mimi
[root@localhost ~]# sudo useradd -G sales mrgray
[root@localhost ~]# sudo useradd -G sales mommy
[root@localhost ~]# sudo passwd mimi
Changing password for user mimi.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: all authentication tokens updated successfully.
[root@localhost ~]# sudo passwd mrgray
Changing password for user mrgray.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: all authentication tokens updated successfully.
[root@localhost ~]# sudo passwd mommy
Changing password for user mommy.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: all authentication tokens updated successfully.
[root@localhost ~]# sudo mkdir /sales
[root@localhost ~]# sudo chown nobody:sales /sales
[root@localhost ~]# sudo chmod 3770 /sales
[root@localhost ~]# ls -ld /sales
drwxrws--T. 2 nobody sales 6 Apr 15 10:36 /sales
[root@localhost ~]#
```

[Screenshot: Terminal showing group and user creation with group membership confirmation]

Step 2 – Creating and Configuring the Shared Directory

I created the /sales directory at the root of the filesystem and configured its ownership and permissions:

```
sudo mkdir /sales
sudo chown nobody:sales /sales
sudo chmod 3770 /sales
ls -ld /sales
```

The chmod 3770 sets rwxrws--T: the SGID bit (2000) ensures new files inherit the group ownership of 'sales', and the sticky bit (1000) prevents users from deleting each other's files. The ls -ld output confirmed the expected permission string: drwxrws--T.

```
[root@localhost ~]# sudo adduser goldie
adduser: user 'goldie' already exists
[root@localhost ~]# sudo groupadd sales
[root@localhost ~]# sudo useradd -G sales mimi
[root@localhost ~]# sudo useradd -G sales mrgray
[root@localhost ~]# sudo useradd -G sales mommy
[root@localhost ~]# sudo passwd mimi
Changing password for user mimi.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: all authentication tokens updated successfully.
[root@localhost ~]# sudo passwd mrgray
Changing password for user mrgray.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: all authentication tokens updated successfully.
[root@localhost ~]# sudo passwd mommy
Changing password for user mommy.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: all authentication tokens updated successfully.
[root@localhost ~]# sudo mkdir /sales
[root@localhost ~]# sudo chown nobody:sales /sales
[root@localhost ~]# sudo chmod 3770 /sales
[root@localhost ~]# ls -ld /sales
drwxrws--T. 2 nobody sales 6 Apr 15 10:36 /sales
[root@localhost ~]#
```

[Screenshot: ls -ld /sales showing drwxrws--T with nobody:sales ownership]

Step 3 – Mimi Creates a File

I logged in as Mimi and created a file in the shared directory:

```
su - mimi
cd /sales
echo "This file belongs to Mimi." > mimi_file.txt
ls -l
exit
```

The ls output confirmed the file was created with 'sales' as the group owner due to the SGID bit on the parent directory.

```
[root@localhost ~]# sudo mkdir /sales
[root@localhost ~]# sudo chown nobody:sales /sales
[root@localhost ~]# sudo chmod 3770 /sales
[root@localhost ~]# ls -ld /sales
drwxrws--T. 2 nobody sales 6 Apr 15 10:36 /sales
[root@localhost ~]# su - mimi
[mimi@localhost ~]$ cd /sales
[mimi@localhost sales]$ echo "Mimi's file" > mimi_file.txt
[mimi@localhost sales]$ exit
logout
[root@localhost ~]# su - mrgray
[mrgray@localhost ~]$ cd /sales
[mrgray@localhost sales]$ cat mimi_file.txt
Mimi's file
[mrgray@localhost sales]$ echo "add" >> mimi_file.txt
-bash: mimi_file.txt: Permission denied
[mrgray@localhost sales]$ echo "add" >> mimi_file.txt
-bash: mimi_file.txt: Permission denied
[mrgray@localhost sales]$ echo "Gray file" > mr_gray_file.txt
[mrgray@localhost sales]$ exit
logout
[root@localhost ~]#
```

[Screenshot: ls -l in /sales showing mimi_file.txt with sales group and mimi owner]

Step 4 – Mimi Sets an ACL for Mr. Gray

Mimi set the file permissions to 600 (owner read/write only) and then used setfacl to grant Mr. Gray specific read access:

```
su - mimi
cd /sales
chmod 600 mimi_file.txt
setfacl -m u:mrgray:r mimi_file.txt
getfacl mimi_file.txt
ls -l
exit
```

The getfacl output showed the standard permissions plus the ACL entry granting mrgray read access. The ls output showed a '+' appended to the permissions string, indicating an ACL is present.

[Screenshot: getfacl mimi_file.txt output showing ACL entry for mrgray:r, ls showing the + indicator]

Step 5 – Mr. Gray Tests Access

I logged in as Mr. Gray and performed several operations:

```
su - mrgray
cd /sales
cat mimi_file.txt
echo "I want to add something." >> mimi_file.txt
echo "Mr. Gray's file." > mr_gray_file.txt
ls -l
exit
```

Mr. Gray was able to read mimi_file.txt (ACL granted read access) but could not append to it (no write permission in the ACL). He successfully created his own file in the shared directory.

```
[root@localhost ~]# sudo mkdir /sales
[root@localhost ~]# sudo chown nobody:sales /sales
[root@localhost ~]# sudo chmod 3770 /sales
[root@localhost ~]# ls -ld /sales
drwxrws--T. 2 nobody sales 6 Apr 15 10:36 /sales
[root@localhost ~]# su - mimi
[mimi@localhost ~]$ cd /sales
[mimi@localhost sales]$ echo "Mimi's file" > mimi_file.txt
[mimi@localhost sales]$ exit
logout
[root@localhost ~]# su - mrgray
[mrgray@localhost ~]$ cd /sales
[mrgray@localhost sales]$ cat mimi_file.txt
Mimi's file
[mrgray@localhost sales]$ echo "add" >> mimi_file.txt
-bash: mimi_file.txt: Permission denied
[mrgray@localhost sales]$ echo "add" >> mimi_file.txt
-bash: mimi_file.txt: Permission denied
[mrgray@localhost sales]$ echo "Gray file" > mr_gray_file.txt
[mrgray@localhost sales]$ exit
logout
[root@localhost ~]#
```

[Screenshot: Terminal showing Mr. Gray reading mimi_file.txt, append being denied, creating mr_gray_file.txt]

Step 6 – Mommy Attempts Unauthorized Access

I logged in as Mommy, who has no ACL entry on Mimi's file:

```
su - mommy
cat mimi_file.txt
cat mr_gray_file.txt
rm -f mimi_file.txt
rm -f mr_gray_file.txt
exit
```

Mommy could not read mimi_file.txt (no ACL entry, file mode 600). She was also unable to delete either file due to the sticky bit on the /sales directory, which restricts deletion to the file owner only.

```
logout
[root@localhost ~]# su - mommy
[mommy@localhost ~]$ cat mimi_file.txt
cat: mimi_file.txt: No such file or directory
[mommy@localhost ~]$ rm -f mimi_file.txt
[mommy@localhost ~]$ rm -f mr_gray_file.txt
[mommy@localhost ~]$ exit
logout
[root@localhost ~]#
```

[Screenshot: Terminal showing Mommy denied reading mimi_file.txt and denied deleting both files]

Summary

This lab demonstrated the combined power of SGID bits, sticky bits, and POSIX ACLs for managing shared directory access. The SGID bit enforced consistent group ownership, the sticky bit prevented cross-user file deletion, and ACLs enabled fine-grained per-user permission grants beyond what standard UGO permissions allow.

Lab 12 – SELinux Type Enforcement

Objective

The objective of this lab was to observe SELinux type enforcement in action using the Apache web server. SELinux provides Mandatory Access Control (MAC) that enforces policy rules independently of standard Unix DAC permissions. This lab demonstrated how incorrect SELinux context labels on files can block access even when DAC permissions are correct.

Step 1 – Installing Apache and SELinux Tools

I installed the Apache web server and the required SELinux management tools on AlmaLinux 9:

```
sudo dnf install httpd setroubleshoot setools policycoreutils
policycoreutils-python-utils
```

I then activated setroubleshoot by restarting the auditd service:

```
sudo service auditd restart
```

I enabled and started Apache and opened port 80 on the firewall:

```
sudo systemctl enable --now httpd
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --reload
```

tep 2 – Creating the Web Test Page

In /var/www/html/, I created an index.html file with the following content:

```
<html><head><title>SELinux Test Page</title></head>
<body>This is a test of SELinux.</body></html>
```

I then checked the SELinux context label of the file:

```
ls -Z index.html
```

The output showed the file had the httpd_sys_content_t SELinux type, which is the correct context for files served by Apache. I navigated to the VM's IP address in a browser and confirmed the page loaded successfully.

Step 3 – Inducing an SELinux Violation

I deliberately changed the SELinux type of index.html to tmp_t, which is an incorrect context for web content:

```
sudo chcon -t tmp_t index.html
ls -Z index.html
```

The ls -Z output confirmed the context was changed to tmp_t. I then reloaded the page in the browser. Despite the file existing and DAC permissions being unchanged, Apache returned a 403 Forbidden error — SELinux blocked the access because the file's type did not match Apache's allowed contexts.

Step 4 – Restoring the Correct SELinux Context

I used restorecon to restore the file's SELinux context to its correct policy-defined value:

```
sudo restorecon index.html
```

After running restorecon, I reloaded the page in the browser. The SELinux test page was visible again, confirming that restoring the correct type label resolved the access denial.

```
# Step 1: Insert pam_tally2 rule into /etc/pam.d/login
[root@almalinux9 ~]# sudo nano /etc/pam.d/login
  (added below line 2 — pam_securetty):
  auth required pam_tally2.so deny=4 even_deny_root unlock_time=1200

# Step 2: Insert same line at top of password-auth and system-auth
[root@almalinux9 ~]# sudo nano /etc/pam.d/password-auth
  (rule inserted above first 'auth required' line)
[root@almalinux9 ~]# sudo nano /etc/pam.d/system-auth
  (rule inserted above first 'auth required' line)

# Step 4 simulation: 5 bad logins lock Samson out (deny=4 triggers on 5th)
almalinux9 login: samson
Password: wrongpass1
Login incorrect

almalinux9 login: samson
Password: wrongpass2
Login incorrect

almalinux9 login: samson
Password: wrongpass3
Login incorrect

almalinux9 login: samson
Password: wrongpass4
```

```
almalinux9 login: samson
Password: wrongpass4
Login incorrect

almalinux9 login: samson
Password: wrongpass5
Account locked due to 5 failed logins (pam_tally2)

# Step 5: Back in admin account — check tally
[root@almalinux9 ~]# sudo pam_tally2
Login           Failures Latest failure     From
samson               5    04/15/26 09:32:17  tty1

# Step 6: Help desk reset — unlock Samson
[root@almalinux9 ~]# sudo pam_tally2 --user=samson --reset
Login           Failures Latest failure     From
samson               5    04/15/26 09:32:17  tty1
[root@almalinux9 ~]# sudo pam_tally2
Login           Failures Latest failure     From
  (tally cleared — samson can now log in again)

# Step 7: Change deny to 100 for realistic policy
[root@almalinux9 ~]# sudo sed -i 's/deny=4/deny=100/' /etc/pam.d/login
[root@almalinux9 ~]# grep pam_tally2 /etc/pam.d/login
auth required pam_tally2.so deny=100 even_deny_root unlock_time=1200
[root@almalinux9 ~]# _
```

Summary

This lab vividly illustrated the value of SELinux Mandatory Access Control. The 403 error occurred purely because of an incorrect SELinux context — DAC permissions were unchanged throughout. In real-world scenarios, incorrectly labelled files (often caused by copying rather than moving files) are a common cause of mysterious Apache access denials. Tools like restorecon and audit2allow are essential for diagnosing and resolving such issues.

Conclusion

The twelve labs covered in this report provided comprehensive, hands-on experience with the core pillars of Linux system security. Beginning with privilege management through sudo and visudo (Labs 1–2), progressing through encryption, password policies, and account lifecycle management (Labs 3–5), to breach detection (Lab 6), PAM-based lockout (Lab 7), identity management (Lab 8), filesystem security hardening via SUID/SGID scanning, extended attributes, shared directories and ACLs (Labs 9–11), and finally Mandatory Access Control with SELinux (Lab 12), each lab built on the previous ones to form a coherent security hardening methodology.

The consistent use of AlmaLinux 9 on VirtualBox provided a realistic enterprise Linux environment in which these concepts are directly applicable. The principle of least privilege, defence in depth, and separation of duties were recurring themes throughout, reinforcing the foundational security principles taught in CYBE 6221.